Git & GitHub

Complete Personal Learning Guide

From zero to pull requests — Windows Edition, Python-first

FOR	Suhas — Aspiring AI/Python Developer
OS	Windows (Git Bash + VS Code)
SCOPE	Setup · Commits · Branches · PRs · GitHub Actions · Claude Workflow
SOURCE	Official GitHub Docs (docs.github.com) — April 2026

01

CHAPTER 1

What is Git & GitHub?

Understanding the tools before you use them

Git vs GitHub — They are not the same thing

A lot of beginners use the words Git and GitHub interchangeably. They are related but distinct. Understanding the difference saves a lot of confusion later.

GIT	GITHUB
Software installed on your computer	A website (cloud service) that hosts Git repositories
Tracks file changes locally — like a time machine for your code	Lets you back up, share, and collaborate on code online
Works completely offline	Requires an internet connection
Free, open-source, created by Linus Torvalds (2005)	Owned by Microsoft, free tier available

The Mental Model — How Git Thinks

Git does not store files — it stores snapshots of changes over time. Every time you make a commit, Git takes a photo of what all your files look like at that moment and stores a reference to that snapshot. This is what allows you to go back to any point in history.

The Three Zones of Git

ZONE	NAME	WHAT LIVES HERE
1	**Working Directory**	Your actual files on disk — what you see in File Explorer
2	**Staging Area (Index)**	A preparation zone — files you've marked to include in the next commit
3	**Repository (.git folder)**	The permanent history — committed snapshots stored in .git/

■ TIP Think of it this way: Working Directory = your desk. Staging Area = your outbox tray. Repository = the filing cabinet. You draft on your desk, put things in the tray, then file them away permanently.

Key Vocabulary

TERM	MEANING
`Repository (repo)`	A folder that Git is tracking. Contains all your files + the full history.
`Commit`	A saved snapshot. Like a save point in a video game.
`Branch`	A parallel version of your code. The default is called main.

`Clone`	Download a copy of a remote repository to your local machine.
`Push`	Upload your local commits to GitHub.
`Pull`	Download the latest changes from GitHub to your machine.
`Pull Request (PR)`	A proposal to merge your branch into another branch — with review.
`Fork`	Your personal copy of someone else's repository on GitHub.
`Merge`	Combine one branch's changes into another.
`HEAD`	A pointer to the commit you're currently on (usually the latest).
`.gitignore`	A file listing things Git should not track (e.g., passwords, big files).

02

CHAPTER 2

Account Setup & Git Install

Windows setup from scratch

Step 1 — Create Your GitHub Account

1 **Go to github.com**
Click Sign Up. Use your personal email (not work). Choose a username that is professional — you will share this with potential employers and collaborators.

2 **Verify your email**
GitHub will send a verification link. Click it. Your account is not fully active until this is done.

3 **Set up your profile**
Add a profile photo, bio, and location. Go to github.com/settings/profile. This matters when others see your PRs.

■ NOTE Choose your username carefully — it appears in every repository URL you create, e.g., github.com/suhas-username/my-python-app

Step 2 — Install Git on Windows

Git does not come pre-installed on Windows. You need Git for Windows, which includes Git Bash — a terminal that lets you run Git commands.

1 **Download Git for Windows**
Go to git-scm.com/download/win — the download starts automatically. This installs Git + Git Bash.

2 **Run the installer**
Accept all defaults. On the screen asking about the default editor, you can choose VS Code if you have it installed.

3 **Verify installation**
Open Git Bash (search in Start menu) and type:

```
git --version
# You should see something like: git version 2.44.0.windows.1
```

■ TIP Always use Git Bash on Windows, not Command Prompt or PowerShell, for running Git commands. It behaves consistently with all documentation.

Step 3 — Configure Git with Your Identity

Git needs to know who you are. This information is attached to every commit you make. Run these two commands in Git Bash — use the same email as your GitHub account:

```
git config --global user.name "Suhas"
git config --global user.email "your@email.com"
```

```
# Set VS Code as your default editor for Git messages:
git config --global core.editor "code --wait"

# Verify everything is set correctly:
git config --list
```

■ REMEMBER Use the same email address you used to sign up on GitHub. Otherwise your commits will not be linked to your GitHub profile.

Step 4 — Authenticate with GitHub (HTTPS Token)

When you push code to GitHub, it needs to verify you are authorised. The modern approach is a Personal Access Token (PAT). GitHub no longer accepts your account password.

1 **Generate a token**
On GitHub: Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token.

2 **Select scopes**
Tick repo (full control of private repositories). Set expiration to 1 year.

3 **Copy and save the token**
You will only see it once. Save it in a notes app temporarily.

4 **Use it as your password**
The first time you push, Git Bash will ask for your GitHub username and password. Enter your username, and paste the PAT as the password.

■ NOTE Windows will remember this token in Credential Manager. You should only need to enter it once per machine.

03

CHAPTER 3

Your First Repository

Create, clone, and make your first commit

Creating a Repository on GitHub

You can start a repository in two ways: create it on GitHub first (recommended) or create it locally and push it up. The GitHub-first approach is simpler for beginners.

1 **Click the + button on GitHub**
Top-right on GitHub → New repository.

2 **Fill in the details**
Repository name: my-python-projects. Visibility: Public (so it builds your portfolio). Tick 'Add a README file'. Tick 'Add .gitignore' → choose Python template.

3 **Click Create repository**
GitHub creates the repo with an initial commit containing README.md and .gitignore.

4 **Clone it to your machine**
Click the green Code button → HTTPS → copy the URL. Then in Git Bash:

```
# Navigate to where you want to store your projects
cd C:/Users/YourName/Documents

# Clone the repository (downloads it to your machine)
git clone https://github.com/your-username/my-python-projects.git

# Move into the cloned folder
cd my-python-projects

# Check the status
git status
# On branch main
# nothing to commit, working tree clean
```

Understanding the .gitignore File

The Python .gitignore template that GitHub added tells Git to ignore common files that should never be committed — like compiled bytecode, virtual environments, and IDE settings. Here is what the key entries mean:

```
# Python compiled files — you never need these in version control
__pycache__/
*.pyc

# Virtual environment folder — never commit this
.venv/
venv/
```

```
# Environment variables — contains secrets, NEVER commit
.env

# VS Code settings folder
.vscode/
```

REMEMBER Never commit your .env file or any file containing API keys, passwords, or tokens. Always add them to .gitignore before your first commit.

A Good README Structure

Every repository should have a README.md — it is the first thing anyone sees. Here is a template you can use for your Python projects:

````
# Project Name

Short one-line description of what this does.

## Setup
```bash
pip install -r requirements.txt
```

## Usage
```bash
python main.py
```

## Built with Claude
Portions of this code were generated with Claude (Anthropic).
All code has been reviewed and tested by the author.
````

04

CHAPTER 4

The Daily Git Cycle

Stage → Commit → Push — your core workflow

The Four Commands You Will Use Every Day

Once your repository is set up, your daily work follows a simple loop. You make changes, tell Git what you want to save, write a message explaining why, and upload it to GitHub.

```
# 1. Check what has changed
git status

# 2. Stage specific files
git add filename.py

# 2b. Stage ALL changed files at once
git add .

# 3. Commit with a message
git commit -m "Add user authentication function"

# 4. Push to GitHub
git push
```

How to Write a Good Commit Message

A commit message is a note to your future self (and anyone else who reads the history). Bad messages make a repository history useless. Good messages make it a valuable log.

BAD ✗	GOOD ✓
`"stuff"`	`"Add input validation to login form"`
`"fix"`	`"Fix divide-by-zero error in calculate_average()"`
`"wip"`	`"WIP: Implement PDF export — incomplete, do not merge"`
`"changes"`	`"Refactor database connection to use connection pool"`

■ TIP Write commit messages in the imperative mood: "Add", "Fix", "Update", "Remove" — not "Added", "Fixed". Think: "If applied, this commit will... [your message]".

Checking History and Undoing Mistakes

COMMAND	WHAT IT DOES
`git log --oneline`	See a compact one-line summary of all commits
`git log -5`	See the last 5 commits with full detail

`git diff`	See exactly what lines changed before staging
`git diff --staged`	See what's staged but not yet committed
`git restore filename.py`	Discard changes to a file (revert to last commit)
`git restore --staged file.py`	Unstage a file (remove from staging, keep changes)
`git commit --amend -m '...'`	Fix the message of the very last commit (before pushing)

REMEMBER git restore is destructive — it throws away your uncommitted changes permanently. Only use it when you are sure you do not want those changes.

05

CHAPTER 5

Branching Strategies

Work on features without breaking main

Why Branches Matter

The main branch of your repository should always contain working, stable code. When you want to add a new feature or experiment with an idea, you create a branch — an isolated copy of the code where you can work freely without affecting main. When the feature is done and tested, you merge the branch back.

■ NOTE Think of main as your production version — the code that runs. Branches are your safe workspaces. GitHub Flow, used by most teams, follows this exact model.

The GitHub Flow (Recommended for Solo and Small Teams)

This is the branching strategy recommended by GitHub's own documentation:

STEP	ACTION	DETAIL
1	**Create a branch**	Branch from main with a descriptive name
2	**Make your changes**	Code, test, commit regularly on your branch
3	**Push the branch**	Push the branch to GitHub
4	**Open a Pull Request**	Propose to merge your branch into main
5	**Review and discuss**	You (or teammates) review the PR
6	**Merge**	Merge the PR into main, delete the branch

Branch Commands

```
# Create a new branch and switch to it immediately
git checkout -b feature/add-user-login

# List all branches (* shows which one you're on)
git branch

# Switch to an existing branch
git checkout main

# Push your new branch to GitHub (first time)
git push -u origin feature/add-user-login

# After first push, just use:
git push

# Delete a branch after it's been merged
git branch -d feature/add-user-login
```

Branch Naming Conventions

Consistent naming makes your repository easy to navigate:

PREFIX	USED FOR	EXAMPLE
`feature/`	New functionality	`feature/pdf-export`
`fix/`	Bug fixes	`fix/null-pointer-crash`
`docs/`	Documentation only	`docs/update-readme`
`chore/`	Maintenance, upgrades	`chore/upgrade-dependencies`
`hotfix/`	Urgent production fix	`hotfix/payment-bug`

06

CHAPTER 6

Pull Requests (PRs)

The heart of GitHub collaboration

What is a Pull Request?

A Pull Request is a formal proposal to merge the changes in your branch into another branch (usually main). It gives you — or collaborators — a chance to review the changes, leave comments, request modifications, and approve before anything is merged. Even when working solo, using PRs builds good habits and creates a clear record of what changed and why.

Creating Your First Pull Request

1 **Push your feature branch**
Make sure all your commits are pushed to GitHub.

2 **GitHub shows a prompt**
After pushing, GitHub.com shows a yellow banner: 'Compare & pull request'. Click it.

3 **Fill in the PR form**
Write a clear title and description. Follow the template below.

4 **Submit**
Click 'Create pull request'.

PR Description Template

```
## What this PR does
- Adds user login with email + password
- Connects to the SQLite database
- Returns JWT token on success

## Why
Required for the authentication module (Issue #12)

## Testing done
- [x] Tested with valid credentials — returns 200 + token
- [x] Tested with wrong password — returns 401
- [x] Tested with missing fields — returns 400

## Notes
Generated with Claude assistance. All logic reviewed and tested manually.
```

Merging the PR

Once you are happy with the PR (or a reviewer approves it), you can merge it. GitHub offers three merge strategies:

STRATEGY	WHAT IT DOES	USE WHEN

Merge commit	Keeps all commits + adds a merge commit	Team projects, want full history
Squash & merge	Combines all PR commits into one clean commit	Solo projects, messy WIP commits (recommended for beginners)
Rebase & merge	Replays commits on top of main — linear history	When you want a clean linear history, advanced use

■ TIP For your personal Python projects, use Squash & merge. It keeps main's history clean — one logical commit per feature, regardless of how many small commits you made while building it.

After Merging — Clean Up

```
# After the PR is merged on GitHub, switch back to main locally
git checkout main

# Pull the merged changes down
git pull

# Delete the feature branch locally (it's been merged, you don't need it)
git branch -d feature/add-user-login
```

07

CHAPTER 7

Resolving Merge Conflicts

When two changes collide

What is a Merge Conflict?

A merge conflict happens when two branches have changed the same line(s) of the same file. Git cannot decide which version to keep, so it marks the file and asks you to decide. Conflicts are normal — they are not errors. Here is what a conflict looks like:

```
<<<<<<< HEAD (your branch)
def greet(name):
return f"Hello, {name}!"
=======
def greet(name):
return f"Hi there, {name}!"
>>>>>>> feature/update-greeting (incoming branch)
```

The section between <<<<<<< and ======= is your current branch's version. The section between ======= and >>>>>>> is the incoming version. You must manually edit the file to pick one (or combine them), then remove all the conflict markers.

Resolving a Conflict — Step by Step

1 **Open the conflicted file**

In VS Code, conflicted files are highlighted in red in the Explorer panel. VS Code also shows Accept Current Change / Accept Incoming Change buttons — you can click them or edit manually.

2 **Decide what the final code should be**

Pick your version, the incoming version, or combine them. Remove ALL the conflict markers (<<<, ===, >>>).

3 **Stage the resolved file**

```
git add filename.py
```

4 **Complete the merge with a commit**

```
git commit -m "Resolve merge conflict in greet function"
```

■ TIP VS Code's built-in merge editor makes conflict resolution much easier visually. When a conflict appears, look for the 'Resolve in Merge Editor' button in VS Code.

■ REMEMBER Never leave conflict markers (<<<, ===, >>>) in your code. If you push a file with conflict markers, your code will be broken.

CHAPTER 8

08 GitHub Actions — CI Basics

Auto-test your Python on every push

What is GitHub Actions?

GitHub Actions is a CI/CD (Continuous Integration / Continuous Delivery) platform built directly into GitHub. It lets you automatically run scripts whenever something happens in your repository — like when you push code or open a PR. For a Python developer, the most valuable first use is automatically running your tests on every push to catch bugs early.

NOTE GitHub-hosted runners already have Python pre-installed. You do not need to install Python on any server. GitHub Actions is free for public repositories.

How GitHub Actions Works

You create a YAML file inside the .github/workflows/ folder of your repository. GitHub reads this file and runs the steps you define whenever the trigger event fires.

```yaml
# .github/workflows/python-ci.yml
name: Python CI

# Trigger: run on every push and every PR targeting main
on:
push:
branches: [ main ]
pull_request:
branches: [ main ]

jobs:
test:
# Use a GitHub-hosted runner with Ubuntu
runs-on: ubuntu-latest

steps:
# Step 1: Check out your code
- uses: actions/checkout@v5

# Step 2: Set up Python
- name: Set up Python
uses: actions/setup-python@v5
with:
python-version: '3.x'

# Step 3: Install your dependencies
- name: Install dependencies
run: |
python -m pip install --upgrade pip
```

```
pip install -r requirements.txt

# Step 4: Run your tests
- name: Run tests
run: python -m pytest
```

Setting Up This Workflow

1 **Create the folder and file**

In your repository, create .github/workflows/python-ci.yml and paste the YAML above.

2 **Add a requirements.txt**

Make sure your project has a requirements.txt listing dependencies. At minimum, add pytest.

3 **Add at least one test**

Create a test_basic.py file with a simple test:

```
# test_basic.py
def test_example():
assert 1 + 1 == 2 # Replace with real tests
```

4 **Push to GitHub**

Push the workflow file. Go to the Actions tab on GitHub to see it running.

■ TIP After the workflow runs, GitHub will show a green tick or red X on your PR. A green tick means all tests passed. A red X means something broke — click it to see the logs.

09

CHAPTER 9

Working with Claude-Generated Code

Save AI code to GitHub cleanly

The Challenge with AI-Generated Code

When you build apps with Claude's help, you get code quickly — but it needs to be managed carefully in Git. The key principles are: review before committing, commit in logical chunks, write honest commit messages, and never commit secrets.

Recommended Workflow: Claude + Git

1 **Start on a feature branch**

Never generate AI code directly on main. Create a branch first.

```
git checkout -b feature/claude-add-pdf-export
```

2 **Have Claude generate the code**

Ask Claude for the feature. Review the output. Understand it before saving it.

3 **Test it locally**

Run the code. Make sure it works. Fix any issues (Claude can help with that too).

4 **Stage and commit with clear attribution**

```
git add pdf_export.py
git commit -m "Add PDF export feature (Claude-assisted)"
```

5 **Open a PR**

Push the branch, open a PR on GitHub, and in the description note which parts were AI-assisted.

What to Include in Your .gitignore for AI Projects

```
# Python standard ignores
__pycache__/
*.pyc
.venv/
venv/

# CRITICAL: Environment variables and API keys
.env
*.env
secrets.py
config_local.py

# Anthropic / OpenAI keys — NEVER commit these
anthropic_key.txt
```

```
api_key.txt

# Large output files
*.pdf
output/
```

REMEMBER If you accidentally commit an API key, treat it as compromised immediately. Revoke the key on the provider's dashboard, generate a new one, and then remove it from Git history using git filter-branch or BFG Repo-Cleaner.

A Good Repository Structure for Python AI Projects

```
my-python-app/
|-- .github/
| |-- workflows/
| |-- python-ci.yml # GitHub Actions CI
|-- src/
| |-- main.py # Main application
| |-- utils.py # Helper functions
|-- tests/
| |-- test_main.py # Tests
|-- .env.example # Template (safe to commit)
|-- .env # Real secrets (in .gitignore!)
|-- .gitignore
|-- README.md
|-- requirements.txt
```

10

CHAPTER 10

Cheat Sheet & Quick Reference

Every command you need, one place

Setup & Configuration

COMMAND	WHAT IT DOES
`git config --global user.name 'Name'`	Set your name (done once)
`git config --global user.email 'x@y.com'`	Set your email (done once)
`git config --list`	View all configuration

Starting & Getting Repositories

COMMAND	WHAT IT DOES
`git init`	Initialize a new Git repository in current folder
`git clone`	Clone a repository from GitHub to your machine
`git remote -v`	Show the remote URLs for this repository

Daily Workflow

COMMAND	WHAT IT DOES
`git status`	Show changed / staged / untracked files
`git add`	Stage a specific file
`git add .`	Stage all changed files
`git commit -m 'message'`	Commit staged changes with a message
`git push`	Push commits to GitHub
`git pull`	Pull latest changes from GitHub
`git log --oneline`	Compact commit history
`git diff`	Show unstaged changes line by line

Branching

COMMAND	WHAT IT DOES
`git branch`	List all local branches
`git branch -a`	List all branches including remote

`git checkout -b`	Create and switch to a new branch
`git checkout`	Switch to an existing branch
`git push -u origin`	Push new branch to GitHub (first time)
`git branch -d`	Delete a merged branch
`git merge`	Merge a branch into your current branch

Undoing Things

COMMAND	WHAT IT DOES
`git restore`	Discard unstaged changes to a file
`git restore --staged`	Unstage a file (keep changes)
`git revert`	Safely undo a commit (creates a new commit)
`git stash`	Temporarily shelve changes to work on something else
`git stash pop`	Restore stashed changes

■ REMEMBER Avoid git reset --hard on shared branches. It rewrites history and causes problems for collaborators. Use git revert instead — it is safe on shared branches.

Useful Inspection Commands

COMMAND	WHAT IT DOES
`git log --oneline --graph`	Visual branch history in terminal
`git show`	Show what changed in a specific commit
`git blame`	See who last changed each line of a file
`git remote show origin`	Detailed info about the remote repository

Further Learning

RESOURCE	URL
Official GitHub Docs	`docs.github.com`
GitHub Flow Guide	`docs.github.com/en/get-started/using-github/github-flow`
GitHub Actions Python CI	`docs.github.com/en/actions/tutorials/build-and-test-code/python`
GitHub Skills (hands-on)	`skills.github.com`
Git Cheat Sheet (Official)	`training.github.com/downloads/github-git-cheat-sheet.pdf`